

Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería
Ingeniería en Sistemas de información y Ciencias de la computación.
Curso - Algoritmos
Ing. Moises Eduardo Velasquez Oliva



Análisis preliminar.

Nombre del alumno: Vivian René Rivera Kraemer

Carné: 0910-26-14699

Lugar y fecha: Jocotenango Sacatepéquez, 15/09/2026

Análisis preliminar:

1. Explicación de la problemática

Para entender bien el reto que representa este proyecto, primero traté de ponerme en los zapatos de la Licenciada Silvia Xitumul y la realidad que vive todos los días en su trabajo. Ella dirige un despacho de contabilidad pequeño en la zona 12 y, junto a sus dos asistentes, lleva los números de 23 clientes que son negocios locales: desde tiendas de barrio hasta talleres mecánicos. El corazón de su trabajo consiste en organizar números en filas y columnas para armar planillas de sueldos, registrar gastos y preparar cotizaciones.

Cualquiera pensaría que hoy en día esto se resuelve fácilmente comprando un programa de computadora, pero la realidad de Silvia es distinta: su pequeño despacho no genera los ingresos suficientes para darse el lujo de comprar licencias oficiales de Microsoft Excel para las tres computadoras que usan.

Al no tener el programa oficial, uno creería que la solución más lógica es usar las opciones gratuitas que existen en internet. Sin embargo, aquí es donde Silvia se topa con su barrera más grande: el servicio de internet en su oficina es sumamente inestable. A menudo se quedan sin conexión durante horas enteras, y, esto casi siempre ocurre en los días de cierre a fin de mes, que es cuando sus clientes más requieren sus reportes. Trabajar en la nube simplemente no es una opción confiable para ella porque la deja paralizada cuando más necesita avanzar.

Esta falta de herramientas le causa varios problemas:

- Cada vez que hay que corregir un solo dato, ella tiene que volver a calcular todo con calculadora, lo cual le toma hasta 40 minutos.
- Al hacer las cosas a mano, es muy fácil equivocarse o arrastrar errores, lo que ya le ha costado dinero.
- Necesita poder personalizar el formato para cada cliente, como poner los saldos negativos en otro color, sin depender de un programa complicado.

¿Por qué el proyecto "HojaClara" es la solución perfecta?

Primero, resolverá su problema de conectividad de raíz. Aunque la aplicación se abrirá usando un navegador web, está diseñada para funcionar completamente desconectada de internet. Silvia podrá hacer sus cierres de mes tranquila, sabiendo que si se cae la red, su programa seguirá funcionando sin interrupciones.

Segundo, le devolveremos esas horas de vida que pierde usando la calculadora. Al crear una cuadrícula donde ella pueda escribir fórmulas, el programa se encargará de hacer la matemática. Si un cliente le pide cambiar el sueldo base a última hora, Silvia solo cambiará ese número y nuestra aplicación actualizará todos los descuentos y totales al instante.

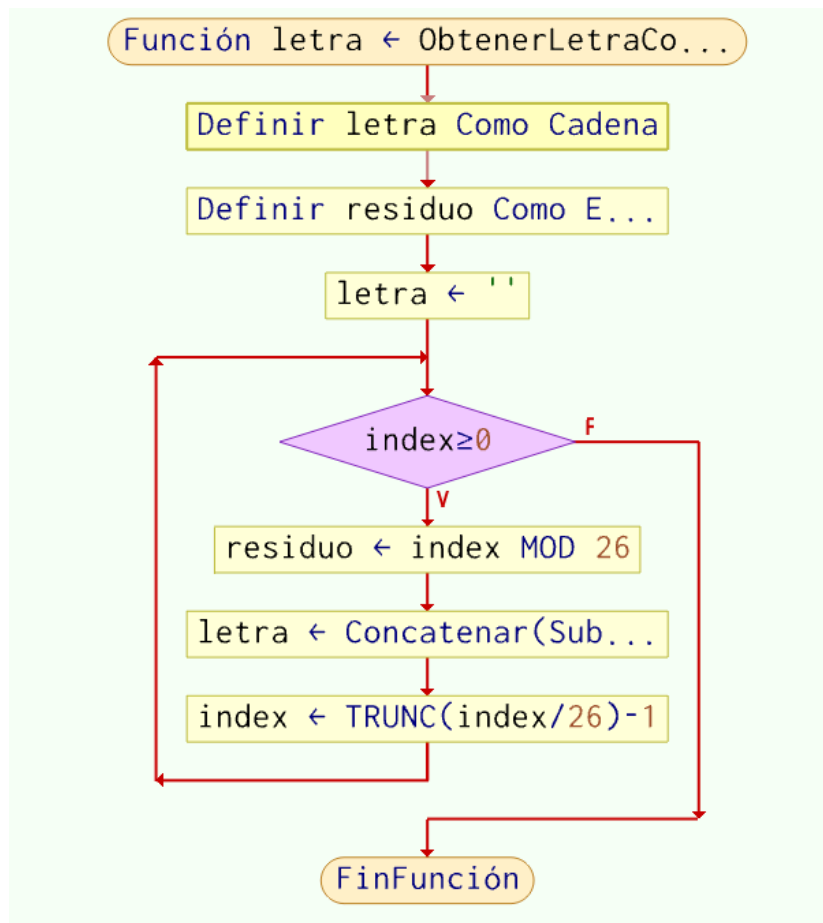
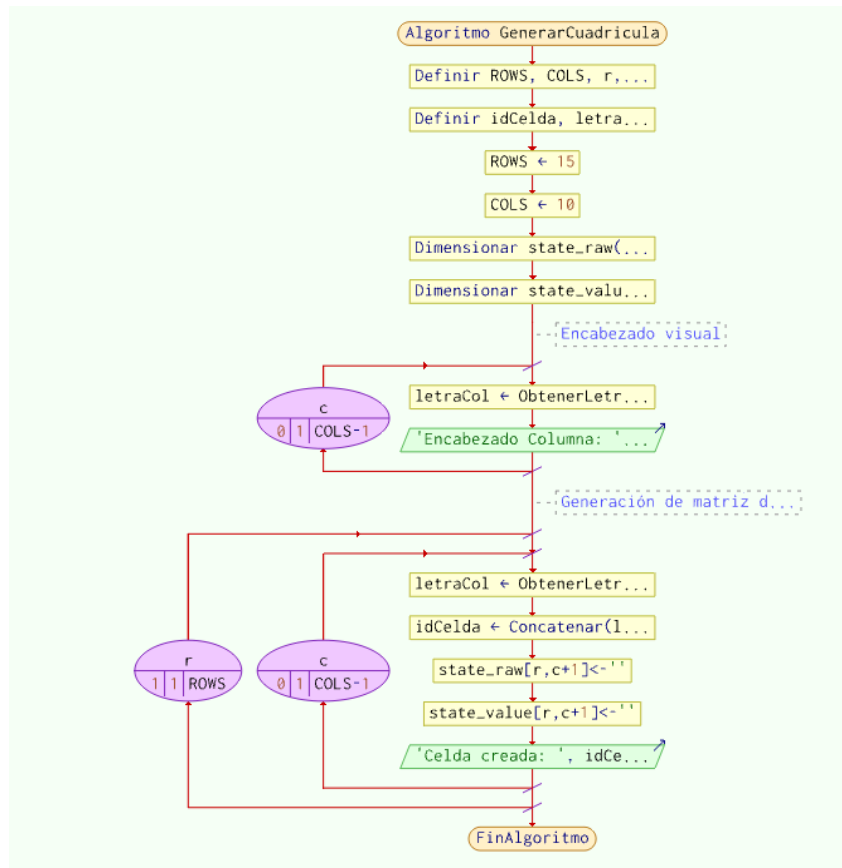
Finalmente, al ser un programa construido por nosotros desde los conceptos más básicos, será totalmente gratuito para ella, no requerirá licencias, no pesará en su computadora y se podrá ir adaptando poco a poco a los colores y formatos que sus clientes le exigen.

2. Lógica de los algoritmos

Al principio, cuando leí todo lo que tenía que hacer el programa, me pareció un problema gigante. Pero aplicando lo que vimos en el curso sobre dividir un problema grande en partes pequeñas, fui pensando en los códigos de PSeInt de la siguiente manera:

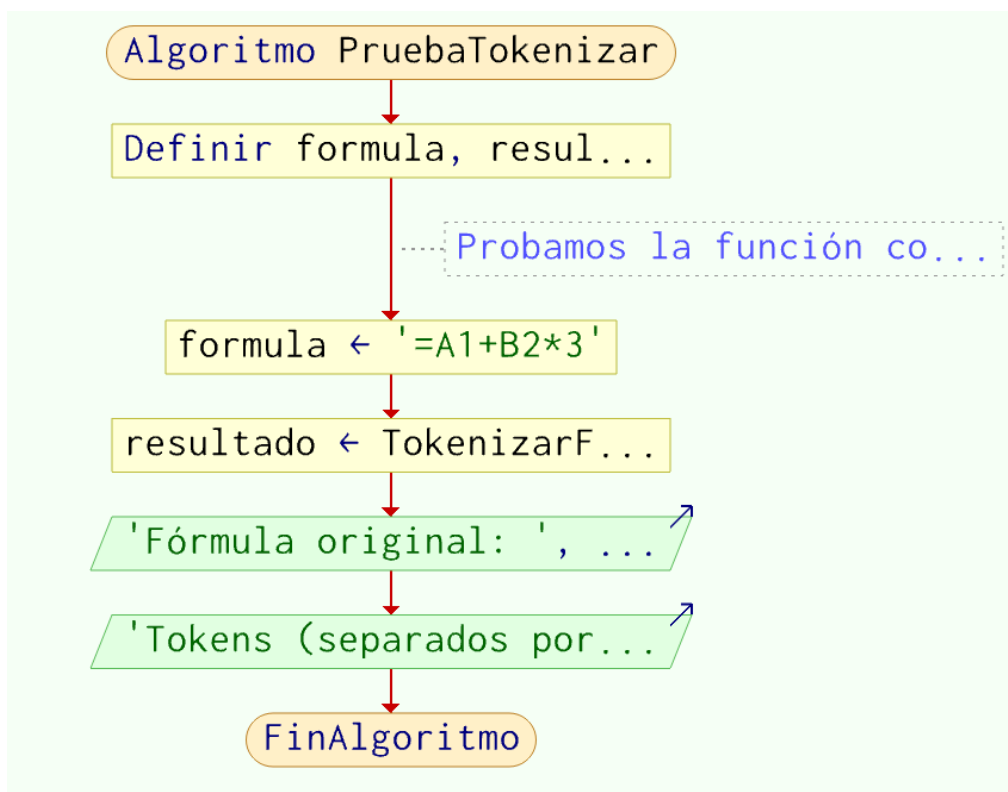
1. Generación de la cuadrícula:

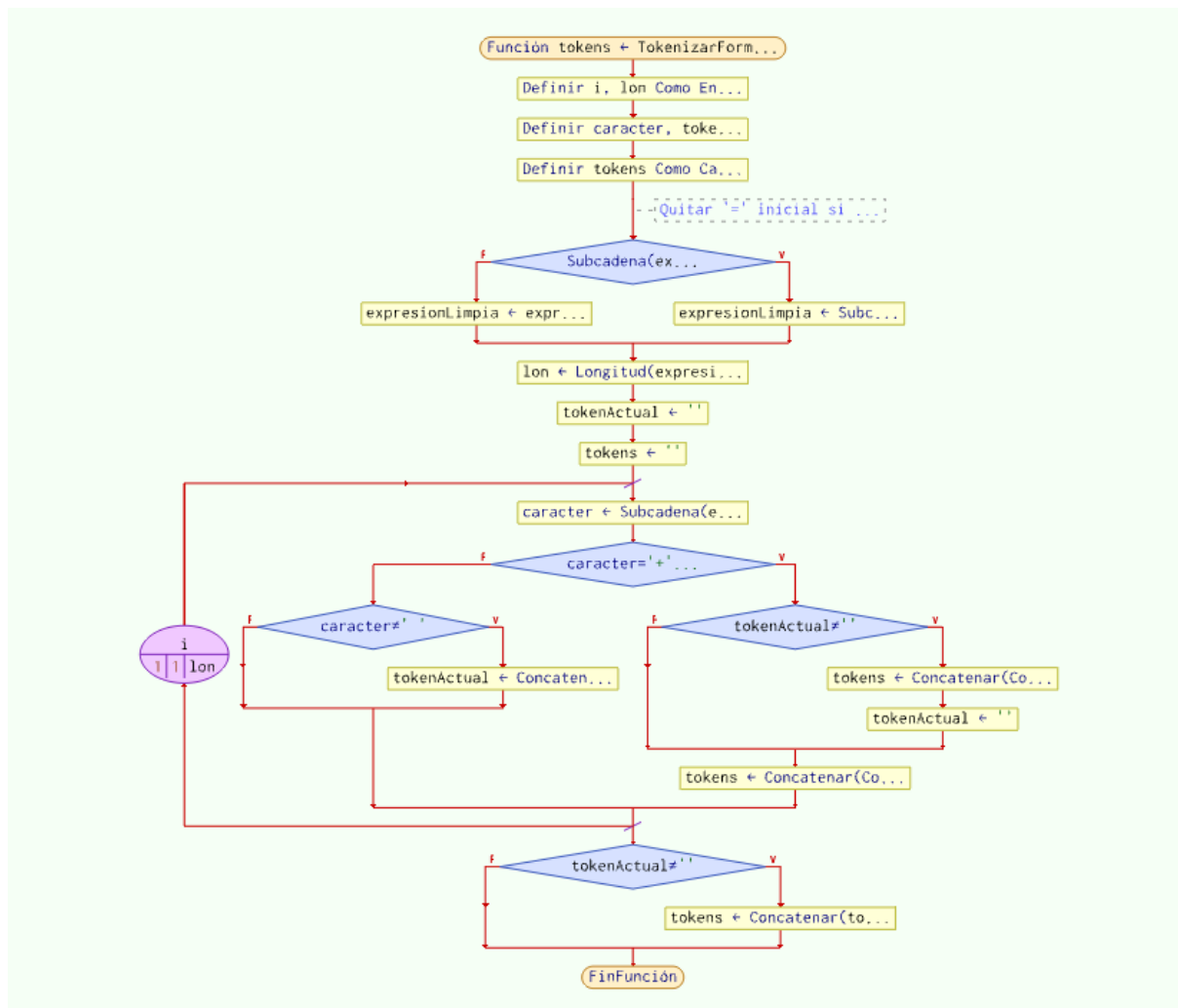
- **Entrada:** Dimensiones de la hoja **ROWS (15)** y **COLS (10)**.
- **Bloque de Proceso 1:** Asignar filas y columnas base. Dimensionar las matrices bidimensionales en memoria (**state_raw** y **state_value**) utilizando las dimensiones definidas.
- **Bucle Externo (Para $r \leftarrow 1$ Hasta ROWS):**
 - **Bucle Interno (Para $c \leftarrow 0$ Hasta COLS - 1):**
 - **Proceso:** Transformar índice c a letra ASCII (**obtenerLetraCol**).
 - **Proceso:** Concatenar el identificador visual **idCelda $\leftarrow$ letra + r** .
 - **Proceso:** Inicializar la celda en las matrices de estado global utilizando coordenadas numéricas: **state_raw[r, c+1] $\leftarrow$ ""** y **state_value[r, c+1] $\leftarrow$ ""**.
 - **Salida / Render:** Imprimir en consola / Crear elemento input con **ID idCelda** en el DOM.
- **Fin:** Cuadrícula inicializada e interactiva.



2. Tokenización de una Fórmula:

- **Entrada:** Cadena *expresion* (ejemplo: " $=A1+B2*3$ ").
- **Decisión / Limpieza:** Si la *expresion* inicia con '=', extraer la subcadena desde la posición 2 en adelante y guardarla en *expresionLimpia*.
- **Bucle (Para $i \leftarrow 1$ Hasta Longitud(*expresionLimpia*)):**
 - **Decisión:** Evaluar el *caracter* actual en la posición *i*:
 - **¿Es Operador (+, -, *, /, (,))?:** Si el acumulador *tokenActual* contiene texto, concatenarlo a la variable *tokens* seguido de una coma "," y limpiar el acumulador. A continuación, concatenar el operador mismo a *tokens*, seguido también de una coma.
 - **¿Es Alfanumérico (y distinto a un espacio en blanco)?:** Acumular el carácter en la variable *tokenActual*.
- **Decisión Final:** Al terminar el bucle, verificar si quedó texto rezagado en *tokenActual*. Si es así, concatenarlo al final de la variable *tokens*.
- **Salida:** Una cadena de texto única con los componentes separados por comas (ejemplo: " $A1,+,B2,*,3$ "), adaptada a las limitaciones de retorno de PSeInt.

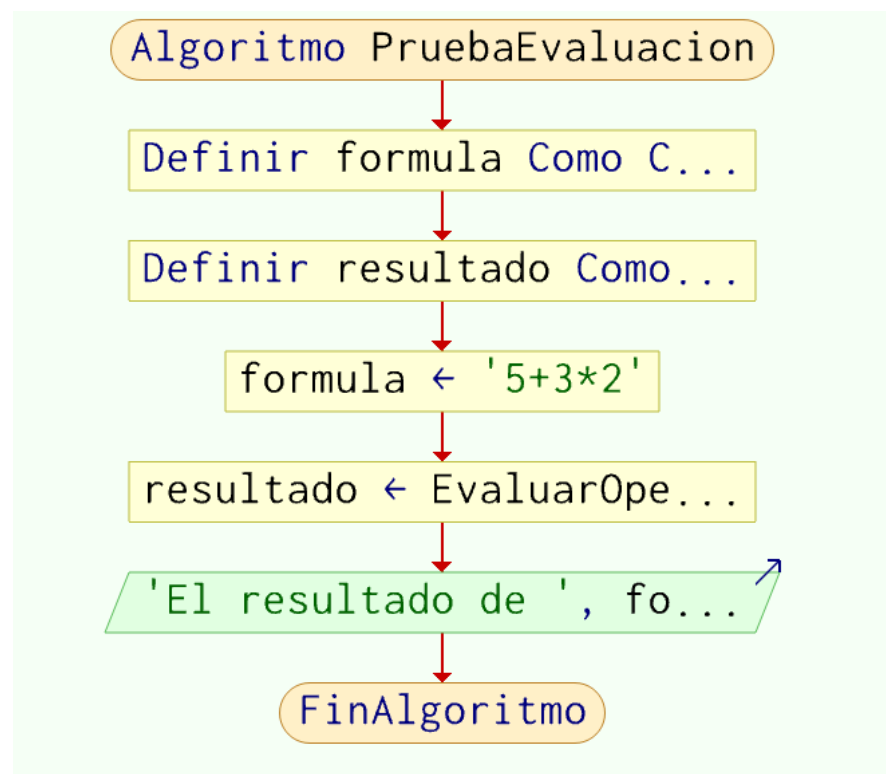


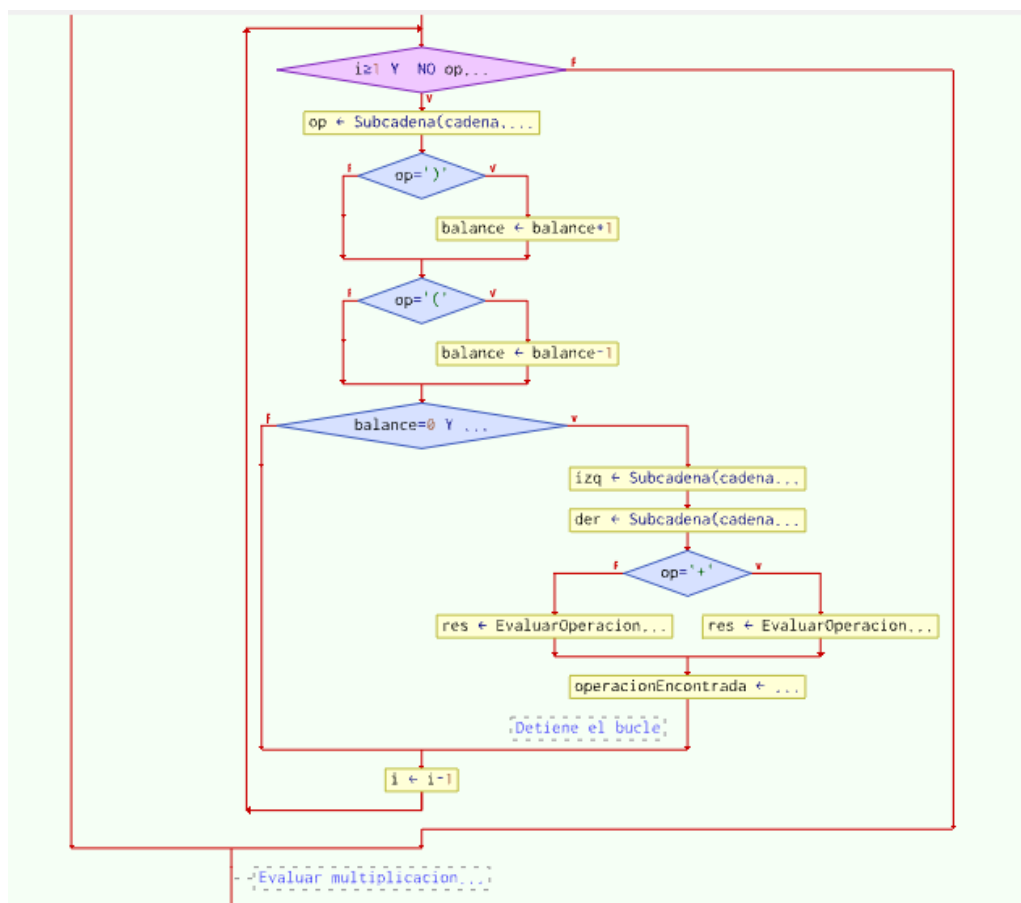
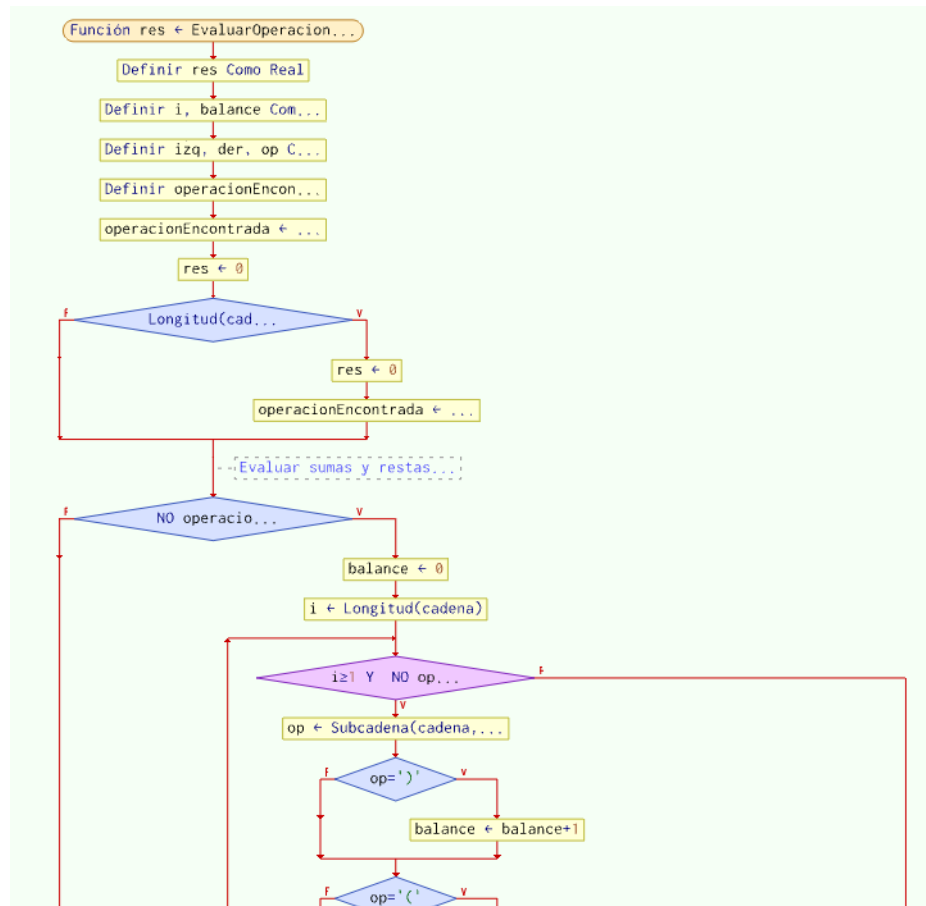


3. Evaluación de la Expresión:

- **Entrada:** Cadena de tokens **cadena**.
- **Inicialización:** Definir una bandera lógica **operacionEncontrada** como Falso.
- **Decisión 1 (Cadena Vacía):** Si la cadena está vacía, asignar 0 al resultado y marcar **operacionEncontrada** como Verdadero.
 - **Bucle 1 (Operadores de Menor Precedencia +, -):**
 - Si no se ha encontrado operación previa, recorrer de derecha a izquierda usando un ciclo iterativo (**Mientras**), respetando el nivel de paréntesis = 0.
 - Si encuentra + o -: Dividir cadena en Izq y Der.
 - Proceso: Asignar al resultado la suma o resta de **evaluarOperaciones(Izq)** y **evaluarOperaciones(Der)**. Marcar **operacionEncontrada** como Verdadero para detener la búsqueda.

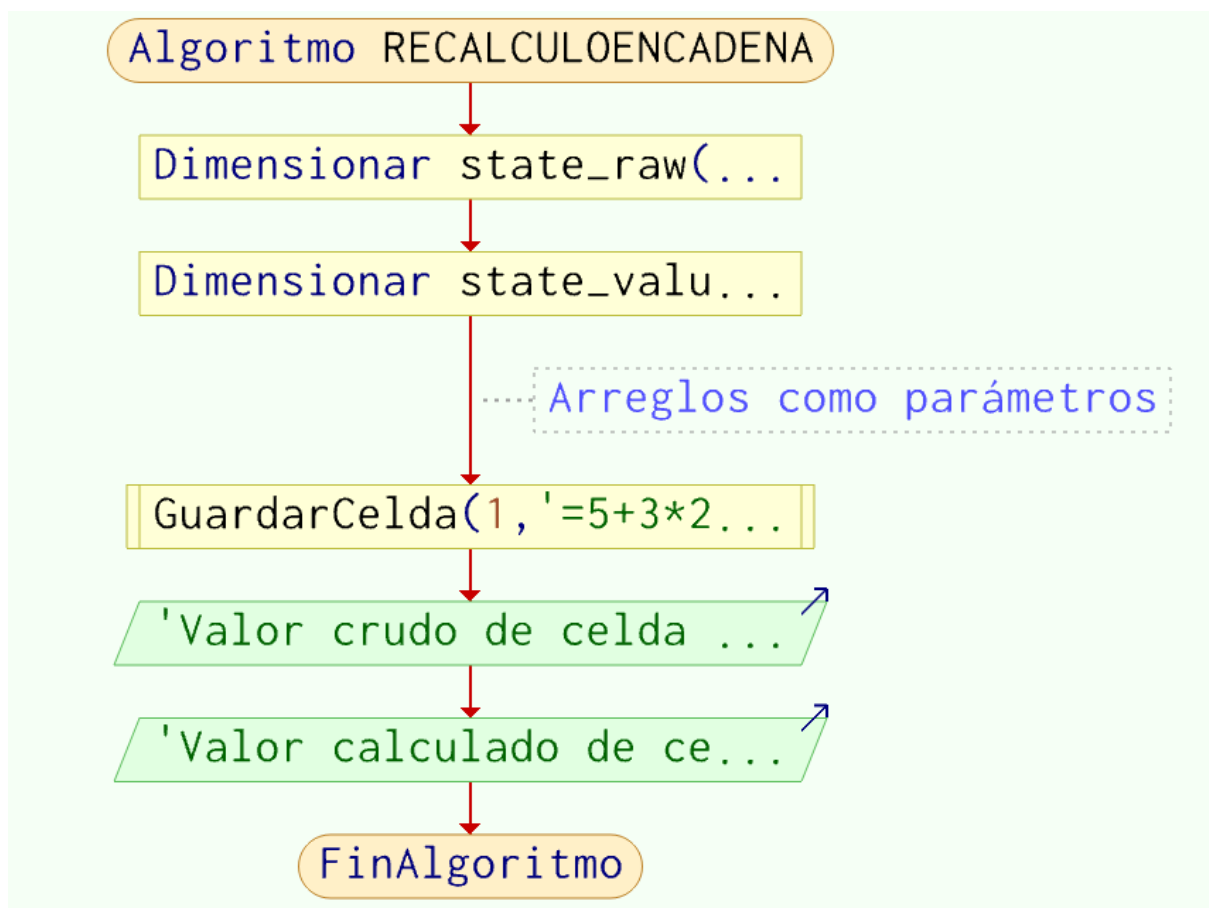
- **Bucle 2 (Operadores de Mayor Precedencia *, /):**
 - Si aún no se ha encontrado operación, recorrer de derecha a izquierda respetando los paréntesis.
 - Si encuentra * o /: Dividir en Izq y Der.
 - **Decisión:** Si es / y la evaluación de Der es 0 -> Imprimir error "#DIV/0!" y asignar 0. Caso contrario, calcular multiplicación/división. Marcar operacionEncontrada como Verdadero.
- **Caso Base:**
 - Si los bucles terminaron y no se encontró ninguna operación, la cadena es un valor final. Convertir la subcadena directamente a valor numérico flotante.
- **Salida:** La función termina de forma natural y retorna la variable de resultado final calculada.

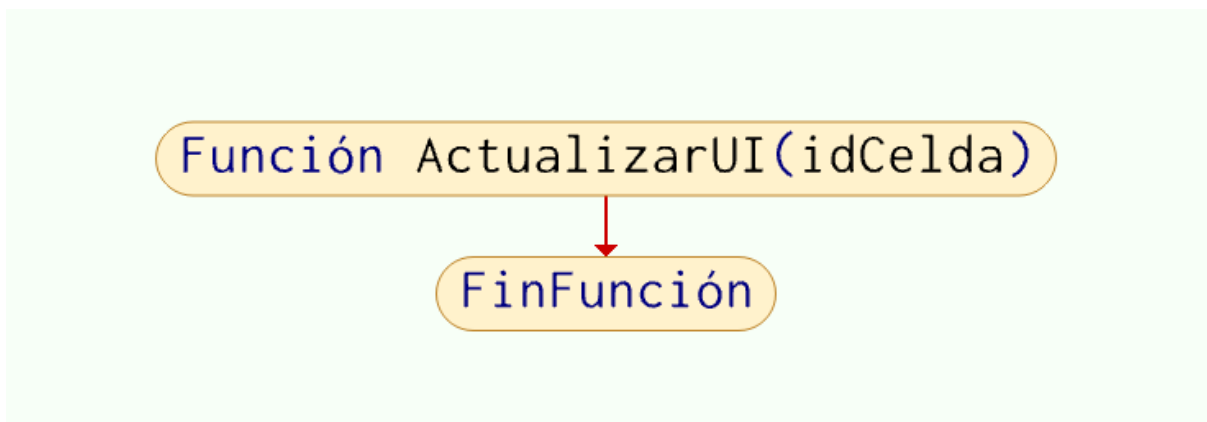
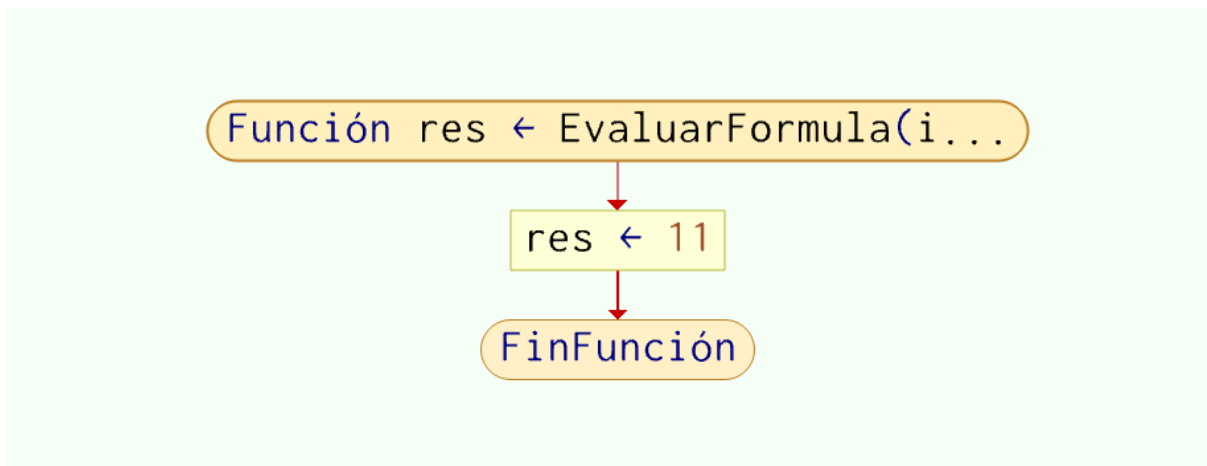
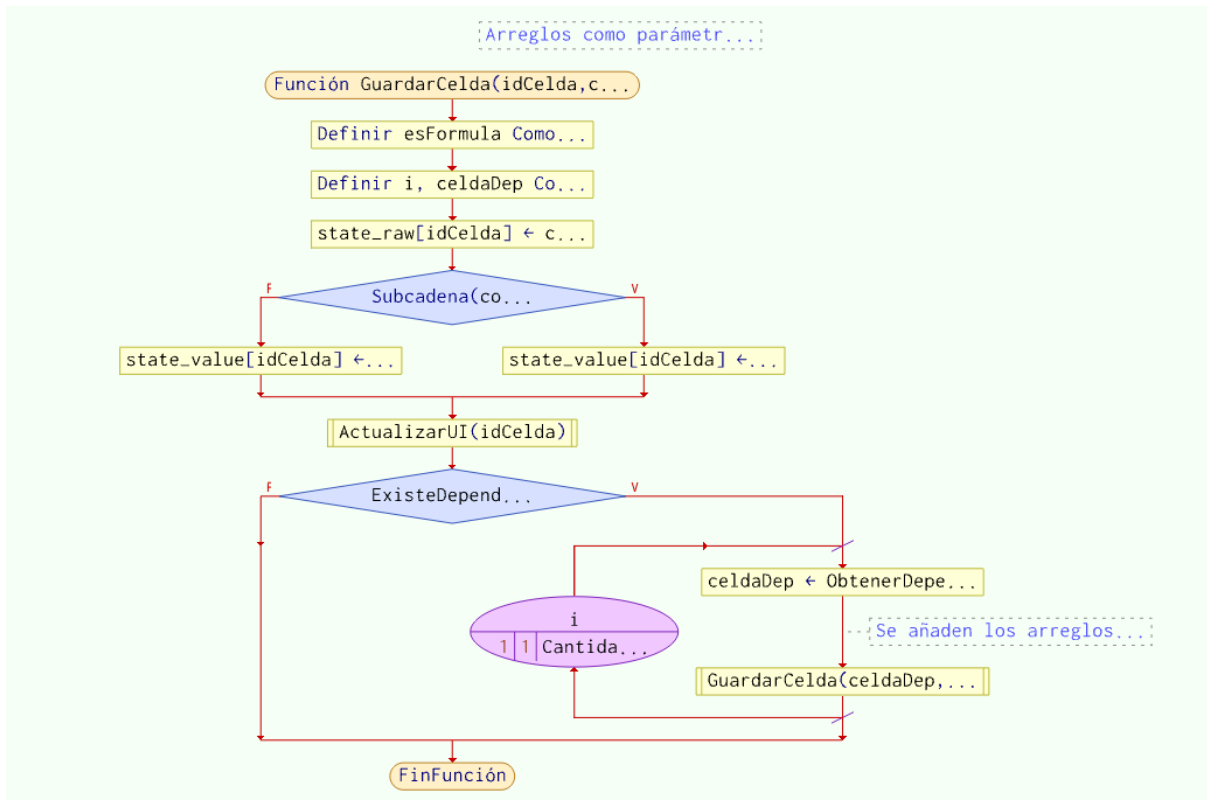




- #### 4. Recálculo en Cadena:

- **Decisión 2 (Validación de dependencias):** ¿La función `ExisteDependencia(idCelda)` retorna Verdadero?
 - **Sí:** Iniciar un bucle **Para** desde 1 hasta `CantidadDependientes(idCelda)`:
 - **Proceso:** Obtener el índice de la celda que depende de la actual (`celdaDep <- ObtenerDependiente(idCelda, i)`).
 - **Llamada Recursiva:** Ejecutar `GuardarCelda(celdaDep, state_raw[celdaDep], state_raw, state_value)` para forzar el recálculo en cascada de la celda dependiente, propagando los arreglos por referencia.
 - **No:** Finalizar el proceso de recálculo de esta rama.





Función existe ← ExisteDepende...

existe ← Falso

FinFunción

Función cant ← CantidadDependi...

cant ← 0

FinFunción

Función dep ← ObtenerDependien...

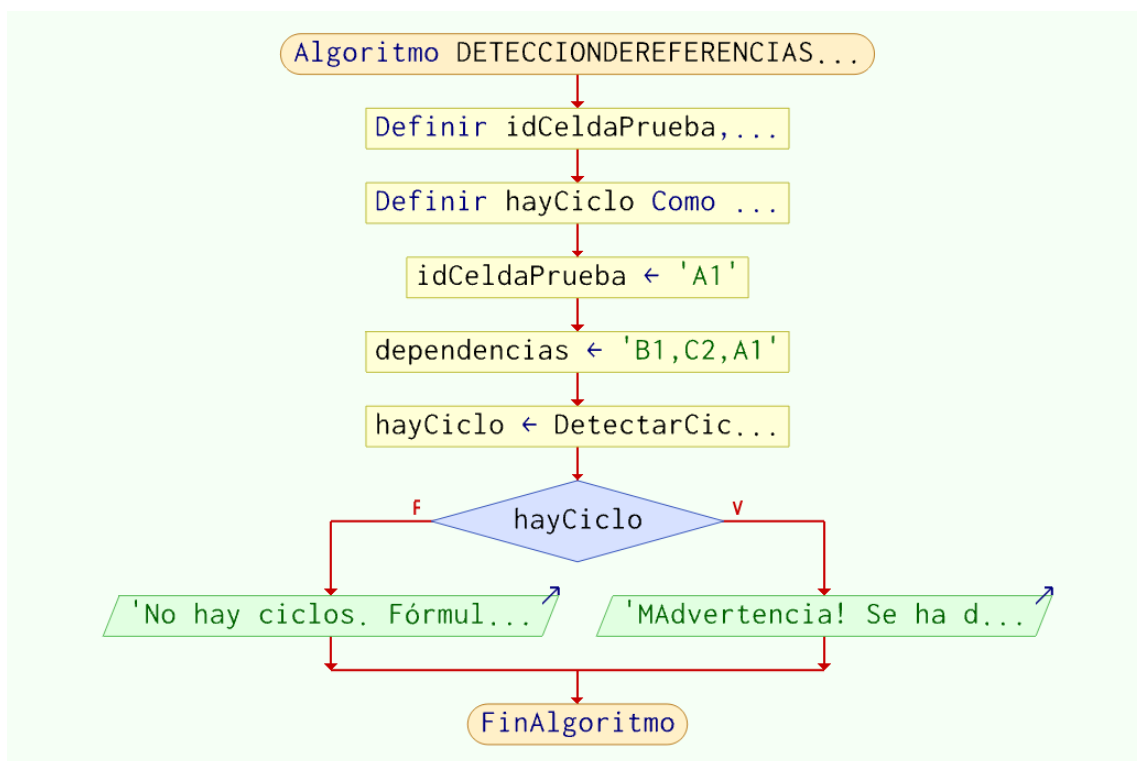
dep ← 0

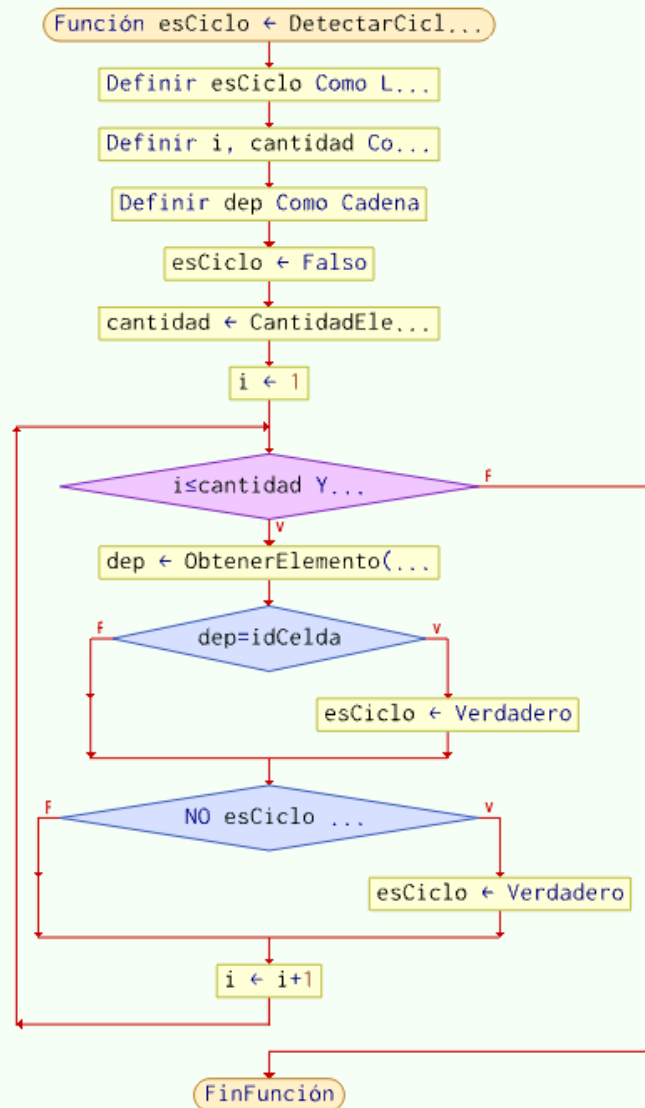
FinFunción

5. Detección de Referencias Circulares:

- **Entrada:** Identificador de la celda a editar `idCelda` y las `dependenciasDetectadas` (lista de referencias que usa la fórmula).
- **Inicialización:** Definir la bandera lógica `esCiclo` como Falso y un contador `i` en 1
- **Bucle** (`Mientras` el índice sea menor a la cantidad de elementos y `esCiclo` sea Falso):

- **Proceso:** Extraer la referencia actual en la variable `dep` usando `ObtenerElemento(dependenciasDetectadas, i)`.
- **Decisión 1 (Auto-referencia):** ¿`dep` es igual a `idCelda`?
 - Sí: Marcar `esCiclo <- Verdadero` (Auto-referencia directa, ej. A1 depende de A1). Esto detendrá el bucle en la siguiente iteración.
- **Decisión 2 (Ciclo Cruzado):** Si aún no hay ciclo, ¿la función `EsDependienteDirecto(idCelda, dep)` retorna Verdadero?
 - Sí: Marcar `esCiclo <- Verdadero` (Ciclo indirecto, ej. A1 depende de B1, y B1 ya dependía de A1).
- **Avance:** Incrementar el contador `i <- i + 1`.
- **Salida:** La función termina de forma natural retornando el valor de la bandera `esCiclo`. Retornará Falso si el bucle terminó sin coincidencias, o Verdadero si se detectó un bucle infinito en algún punto.





Función cant ← CantidadElement...

cant ← 3

FinFunción

Función cant $\leftarrow$ CantidadElement...

cant $\leftarrow$ 3

FinFunción

Función elemento $\leftarrow$ ObtenerElem...

Definir elemento Como ...

indice=1

elemento $\leftarrow$ 'B1'

indice=2

elemento $\leftarrow$ 'C2'

indice=3

elemento $\leftarrow$ 'A1'

FinFunción

Función esDep $\leftarrow$ EsDependienteD...

esDep $\leftarrow$ Falso

FinFunción

Diseño de la Estructura de Datos:

Debido a que el lenguaje estructurado no soporta objetos, diccionarios (mapas clave-valor) o arreglos dinámicos con índices alfanuméricos (como "A1"), el estado de la aplicación se gestiona mediante un enfoque de **arreglos paralelos y tipos primitivos**.

1. Estado Principal de las Celdas (Arreglos Paralelos)

El motor de cálculo utiliza arreglos unidimensionales de tamaño estático (ej. `Dimension[100]`), donde cada índice numérico entero representa una celda única (ej. 1 = A1, 2 = A2).

- `state_raw` (**Arreglo de Cadenas**): Almacena el contenido original ingresado por el usuario sin procesar (ej. "`=5+B2`", "`Hola`").
- `state_value` (**Arreglo de Cadenas/Reales**): Almacena el resultado final de la celda. Si `state_raw` contiene una fórmula, aquí se guarda el resultado evaluado. Si contiene texto plano, se copia el valor.
- *Nota de alcance*: Estos arreglos se definen en el **Algoritmo** principal y se pasan **Por Referencia** a los subprocesos (como `GuardarCelda`) para poder leer y modificar su información en memoria sin violar el encapsulamiento.

2. Listas de Tokens

Dado que las funciones en PSeInt no pueden retornar arreglos, la descomposición de fórmulas (Tokenización) utiliza una **Cadena Única Delimitada**.

- En lugar de generar un arreglo [`"A1"`, `"+"`, `"B2"`], el sistema produce una cadena continua: "`A1,+,B2`". Las funciones de evaluación recorrerán esta cadena separando los elementos cuando sea necesario.

3. Grafo de Dependencias

Para el recálculo en cadena y la detección de ciclos, la relación "quién depende de quién" se abstrae en un modelo de Grafo Dirigido accedido mediante funciones auxiliares.

- `CantidadDependientes(idCelda)` y `ObtenerDependiente(idCelda, i)`: Reemplazan a los mapas dinámicos (`dependencias[id]`). Estas funciones actúan como una interfaz para recorrer las aristas del grafo de dependencias usando bucles iterativos.
- `CantidadElementos(lista)` y `ObtenerElemento(lista, i)`: Se utilizan para recorrer las referencias detectadas dentro de una fórmula específica durante el análisis de dependencias circulares.

4. Control de Flujo Lógico

- En reemplazo de las salidas tempranas (**Retornar** en medio de un bucle), las estructuras de repetición (**Mientras**) dependen de **Variables de Estado Lógicas** (como **operacionEncontrada** o **esCiclo**). Estas actúan como interruptores que cambian a **Verdadero** cuando se halla el resultado, permitiendo que los ciclos terminen ordenadamente y la función alcance su punto de retorno natural.